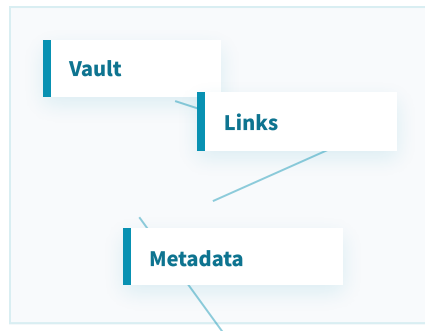


AI로 Obsidian 사용하기



Obsidian은 메모를 쌓아두는 앱이 아니라, 내가 저장한 생각을 AI와 함께 다시 꺼내 쓰는 Second Brain이다.

Vault 구조

Second Brain의 최소 지도

위키링크·백링크

생각의 연결과 맥락

태그

여러 관점의 임시 렌즈

frontmatter

오래 유지할 메타데이터

왜 Obsidian은 AI와 잘 맞는가?

내 Second Brain이 파일·링크·메타데이터로 열려 있기 때문이다

관점	일반 노트 앱	Obsidian Vault
저장 단위	앱 내부 데이터	.md 파일 — 평문으로 읽고 쓴다
접근 방식	복붙 / API 의존	파일 직접 접근 — 폴더 단위 컨텍스트
연결 표현	앱 기능 안에 갇힘	[[wikilink]], Markdown 링크로 노출
메타데이터	DB 필드 중심	YAML frontmatter / Properties
AI 협업	대화창에 참고자료 첨부	Vault 자체가 Second Brain의 작업면이 된다

Vault 구조 — Second Brain이 길을 찾는 최소 지도

폴더는 분류 체계가 아니라 행동 경로다

```
Vault/  
├─ 00-System/      # 운영 규칙 · 템플릿  
├─ 01-Inbox/       # 아직 검증 전인 입력  
├─ 03-Projects/    # 지금 진행 중인 산출물  
├─ 04-Areas/       # 계속 관리할 책임 영역  
├─ 05-Resources/   # 참고 자료  
└─ 07-Knowledge/   # 정제 · 재사용 Knowledge 레이어
```

1

입구와 출구 분리

Inbox와 Knowledge를 섞지 않는다.

2

폴더 의미 문서화

각 루트에 README.md 또는 AGENTS.md를 둔다.

3

이동 규칙 우선

완벽한 분류보다 “언제 Resources에서 Knowledge로 승격되는가?”를 정한다.

폴더는 보관함이 아니라, 사람과 AI가 다음 행동을 판단하는 **routing table**이다.

위키링크와 백링크 — 생각의 맥락을 드러내는 연결

위키링크는 내가 거는 연결이고, 백링크는 시스템이 보여주는 역방향 맥락이다

Wikilink란?

[[노트]] 처럼 내가 먼저 거는 연결이다.

Backlink란?

지금 보는 노트를 어디에서 참조했는지 보여주는 역방향 목록이다.

Backlink의 역할

관련 맥락, 중복 개념, 재사용할 지식을 다시 발견하게 한다.

연결이 쌓일 때 생기는 효과

Wikilink는 내가 먼저 길을 만든다

어떤 개념을 독립 노트로 분리하고, 지금 읽는 흐름에서 어디로 이동할지 의도적으로 지정한다.

Backlink는 그 길들이 모여 생긴 결과다

여러 노트에서 이 노트를 참조할수록, 지금 보는 노트의 활용 맥락과 재사용 지점이 드러난다.

그래서 **backlink**는 “내가 미리 적어 둔 목차”가 아니라, **연결이 축적되며 나중에 발견되는 맥락 지도에 가깝다.**

AI와 함께 쓰는 연결 프롬프트

“이 노트에서 [[wikilink]] 후보 3개를 고르고, 각 링크가 쌓일 때 어떤 backlink 맥락이 생길지 알려줘.”

위키링크 · 백링크 · 섹션 링크 · 임베드 비교

내가 쓰는 링크, 시스템이 보여주는 맥락, 본문 안에 붙이는 자료를 구분해서 본다

핵심 구분: **wikilink**는 사용자가 직접 쓰는 연결 문법이고, **backlink**는 그 결과를 시스템이 역방향으로 보여주는 참조 목록이다. 여기에 **section link**는 위치를 좁히고, **embed**는 내용을 현재 문맥 안으로 가져온다.

구분	무엇인가	어떻게 보이냐	언제 쓰냐
wikilink	사용자가 직접 쓰는 노트 연결 문법	<code>[[검증 대역폭]]</code>	개념 노트, 프로젝트 노트, 관련 아이디어를 의도적으로 연결할 때
backlink	현재 노트를 참조한 다른 노트들의 역방향 목록	Backlinks 패널 · 참조 목록	이 노트가 어디에서 다시 쓰였는지, 어떤 맥락과 연결되는지 볼 때
section link	특정 노트의 heading까지 바로 가는 링크	<code>[[M1#PARAM+K 레이어]]</code>	긴 노트에서 필요한 근거 위치나 핵심 섹션만 정확히 가리킬 때
embed	노트나 미디어를 현재 문서 안에 그대로 포함하는 방식	<code>! [[diagram.png]]</code> <code>! [[회의록]]</code>	다른 노트로 이동하지 않고, 현재 문맥 안에서 내용을 바로 보여주고 싶을 때

기억할 한 줄

wikilink는 내가 쓰는 연결이고, backlink는 시스템이 보여주는 결과이며, section link는 위치를 좁히고, embed는 내용을 안으로 가져온다.

태그 — 여러 관점에서 다시 찾기 위한 임시 렌즈

폴더가 주소라면 태그는 같은 노트를 여러 관점으로 묶는 표시다

태그란?

노트를 한 폴더에 고정하지 않고, 주제·작업·맥락으로 다시 묶기 위한 간단한 표시다.

검색과 필터링

#ai · #pkm · #obsidian

임시 triage

#to-review · #needs-source · #to-link

작업 큐

#to-process · #to-summarize · #follow-up

맥락 표시

#meeting · #lecture · #idea

태그 남용 신호와 처방

태그가 100개 이상

폴더를 복제하고 있다 → 핵심 태그만 유지

같은 의미의 태그 중복

AI 분류가 흔들린다 → 작은 태그 사전 작성

사람 이름·고객명 남발

검색 노이즈가 커진다 → frontmatter 필드로 이동

태그는 영구 분류나 운영 status의 기준값이 아니라, 사람과 AI가 현재 작업을 빠르게 다시 보는 임시·triage 렌즈다.

frontmatter / Properties — 태그와 구분되는 검증 가능한 메타데이터

태그가 임시 렌즈라면, 운영 status와 오래 유지할 사실은 frontmatter에 둔다

```
---
title: "AI로 Obsidian 사용하기"
type: lecture-note
status: draft          # 운영 상태의 기준값
created: 2026-05-08
updated: 2026-05-08
tags:
  - pkm-course
  - obsidian
  - to-link             # 임시 triage만 태그로
related:
  - "[[PARA+K 레이어]]"
  - "[[검증 대역폭]]"
---
```

태그 vs frontmatter

태그는 임시 triage에, frontmatter는 운영 status와 검증 가능한 사실에 쓴다.

1

목적·상태·관계 선포약

본문을 읽기 전에 노트가 무엇인지, 어디에 쓰이는지 먼저 알 수 있다.

2

검증 가능한 필드로 필터링

Dataview/Bases에서 구조화된 목록과 작업 대상을 안정적으로 고를 수 있다.

3

정확해야 할 정보 분리

고객명, 민감도, 출처, 운영 status는 태그 대신 frontmatter에 둔다.

좋은 frontmatter는 AI에게 주는 짧은 업무 인수인계서이자, 운영 status의 기준값을 포함하는 검증 가능한 metadata다.

템플릿 · Guardrail · 루틴 — AI가 일관되게 쓰게 만드는 운용 규칙

빈 노트를 주지 말고, 작성 양식과 검증 절차를 함께 준다

템플릿 전략

프로젝트·회의·지식 노트처럼 유형별 최소 frontmatter와 본문 섹션을 정해, AI가 같은 양식으로 초안을 만들게 한다.

Guardrail

새 폴더·태그·필드를 만들기 전에 기존 구조를 먼저 확인하고, 원본 보존 → 변경 제안 → 사람 검토 순서로 반영하게 한다.

1

READ

관련 노트와 README/AGENTS.md를 먼저 읽게 한다.

2

STRUCTURE

폴더 위치, 제목, frontmatter를 제안하게 한다.

3

LINK

wikilink 후보와 예상 backlink 맥락을 제안하게 한다.

4

VERIFY

사람이 의미·보안·중복을 검토한 뒤 반영한다.

핵심 메시지

템플릿은 입력 양식이고, guardrail은 변경 규칙이며, 루틴은 사람과 AI가 Second Brain을 함께 다루는 작업 순서다.

오늘 남길 것 — AI-Ready Second Brain 최소 운영 세트

완벽한 정리가 아니라, 다시 찾고 다시 쓰고 다시 검증할 수 있는 Vault

1

Vault 루트 지도

5~7개 폴더와 각 폴더의 의미

2

노트 템플릿 1개

최소 frontmatter와 본문 섹션 포함

3

링크·태그 운용 기준

링크 후보 방식과 작은 태그 사전

4

AI 검토 프롬프트

READ → STRUCTURE → LINK → VERIFY

핵심은 AI에게 “정리”를 맡기는 것이 아니라, AI가 읽을 구조·링크·metadata를 먼저 갖추고 그 제안을 사람이 검증 가능한 흐름 안에 두는 것이다. 다음 단계는 이 흐름을 Vault 밖의 웹·디자인 도구까지 확장하는 일이다.

NEXT MODULE

**M5: MCP 연동
디자인·자료조사**